4.10 Danger zone, or know thy SQL

When working with Django `Model` classes, it can be easy to forget that everything is a proxy to the database. The abstraction is helpful and allows you to quickly write Python to solve your data storage needs. The query manager attached to your `Model` allows you to do wonderfully complex queries. All of this is good, right up to the point where it is not.

If you start thinking of a `Model` as a Python object, rather than as a proxy to a table, you could end up with some very inefficient SQL. An extreme example of this happens when you map conceptual objects to a `Model`. Consider a website for a school, where you have students, teachers, and staff. From an object-oriented perspective you might design a student object, a teacher object, and a staff object. Translated into a Django `Model`, that results in three tables. If each of those has a name field, it is easy to make a mistake and have each field be a different length. If you want to add address information, you have to add it in three different places.

A flatter design is better. Instead of students, teachers, and staff, you have a person. The person has a role attribute to indicate whether someone is a student, teacher, or staff. This design results in all the people being in the same table.

This is an overly simplified example, but the object-oriented tools and design techniques you use in Python may not translate well to a database. You need to consider the resulting database structure when thinking about your Django `Model` design.

The impact of the database structure affects the performance of your database. Consider the many-to-many relationship you built in this chapter, between musicians and bands. To query the members of a band, a look-up gets done on the many-to-many table. As Django returns objects, rather than just their IDs, the corresponding band members must also be looked up in the musician table. To fetch a single band and its four members, a query is done to the band table to find the band, a query with four results gets done on the band/musician many-to-many table, and a query gets performed on the musicians table for each of the four musicians. Clever SQL can reduce the performance cost of all this work, and Django does a pretty decent job of completing this for you. The more inter-relationships your objects have, the messier all of this becomes.

For production systems with high volumes, or any system with a lot of data, the underlying SQL can make a huge difference on the performance. The better you understand SQL and what the ORM is doing on your behalf, the better your results. This does not mean you need to be an SQL expert tomorrow, but be aware that although it feels like you're writing Python, you're actually writing SQL.

4.11 Exercises

1. Modify the `musicians` index view to take an additional query parameter that specifies the number of objects per page. Make sure to support reasonable default and maximum values.
2. Using the musician index and detail pages as a reference, create similar pages for your `Band` objects. Inside the template, use the `musicians` relationship to show the members of the band.